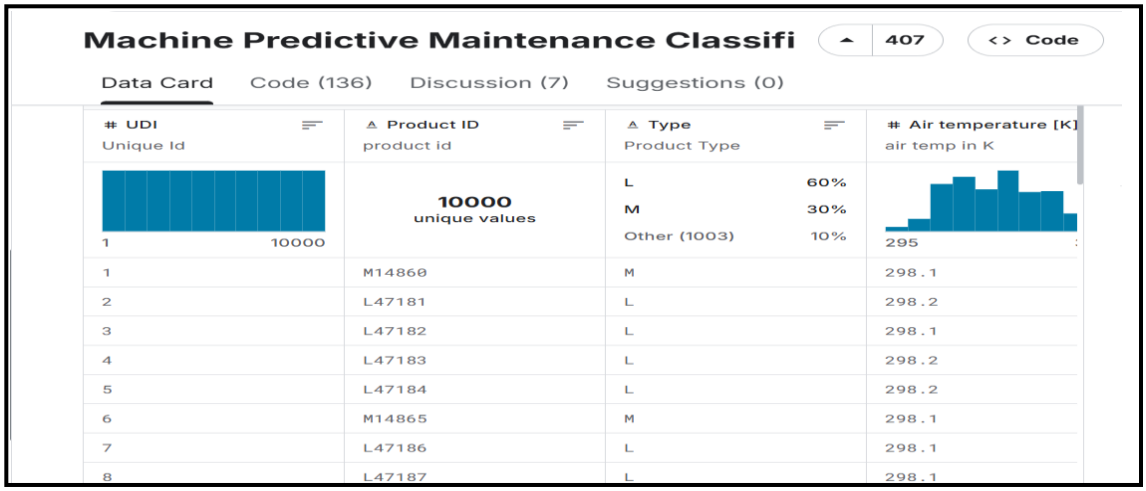
LAB # 02

OBJECTIVE:

To implement a perceptron, analyze its decision boundary, and study its limitations.

LAB TASKS:

KAGGLE DATASET:



1. Go to Kaggle and download a binary classification dataset. train a perceptron to predict the target, and test it on new data points.

CODE:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score
from sklearn.preprocessing import StandardScaler
from google.colab import files
uploaded = files.upload()
data = pd.read_csv("predictive_maintenance.csv")
data.columns = data.columns.str.strip()
data = data.drop(['UDI', 'Product ID'], axis=1)
data['Type'] = data['Type'].map({'L': 0, 'M': 1, 'H': 2})
target_col = [col for col in data.columns if "failure" in col.lower()][0]
X = data.drop(target_col, axis=1)
y = data[target_col]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = Perceptron()
model.fit(X_train, y_train)
y_predict = model.predict(X_test)
print("Model Accuracy:", accuracy_score(y_test, y_predict))
new_input = pd.DataFrame([X.iloc[0].values], columns=X.columns)
new_data_scaled = scaler.transform(new_input)
prediction = model.predict(new_data_scaled)
print("Prediction:", "Machine Failure" if prediction[0] == 1 else "No Failure")
```

OUTPUT:

```
Choose files predictive_...ntenance.csv
predictive_maintenance.csv(text/csv) - 531014 bytes, last modified: 12/04/2026 - 100% done
Saving predictive_maintenance.csv to predictive_maintenance (7).csv
Model Accuracy: 0.9925
Prediction: No Failure
```

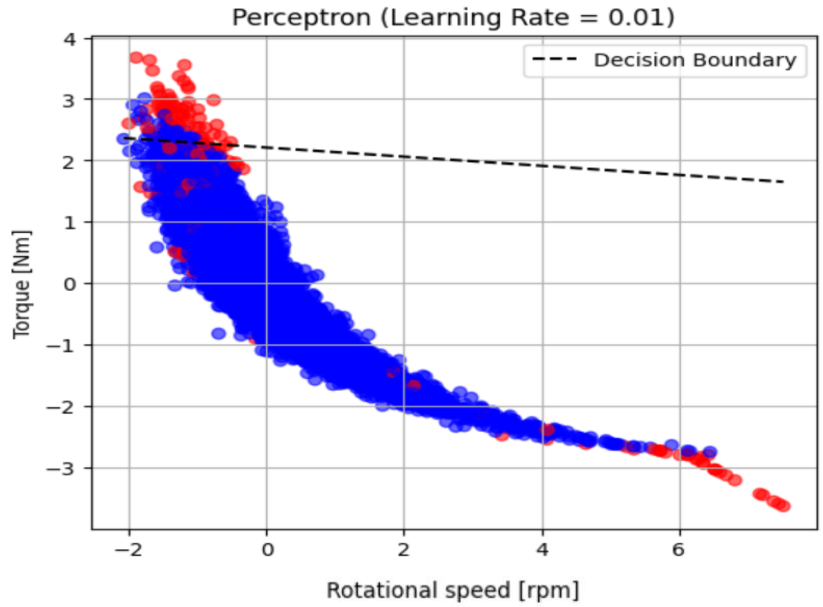
2. Modify the perceptron to train on shuffled data multiple times and plot how the decision boundary changes. Compare results for different learning rates (e.g., 0.01, 0.1) and observe which learning rate gives faster convergence.

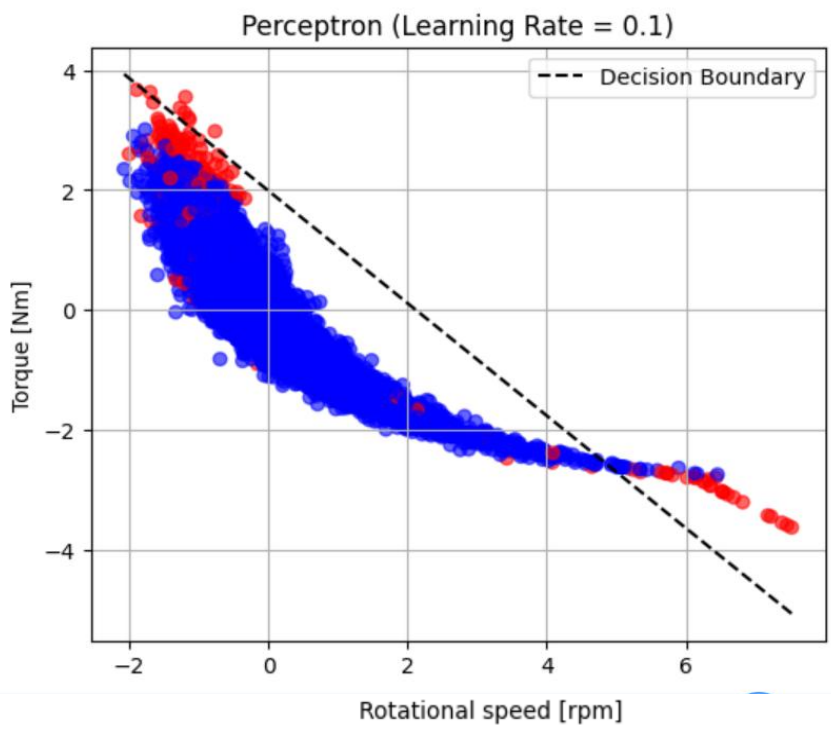
CODE:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from google.colab import files
uploaded = files.upload()
file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)
df.columns = df.columns.str.strip()
df = df.dropna()
y = df["Target"].values
X = df[["Rotational speed [rpm]", "Torque [Nm]"]].values
X = X.astype(float)
X = (X - X.mean(axis=0)) / X.std(axis=0)
def step(z):
    return 1 if z > 0 else 0
class Perceptron:
    def __init__(self, lr=0.01, epochs=20):
        self.lr = lr
        self.epochs = epochs
        X = np.insert(X, 0, 1, axis=1)
        self.w = np.zeros(X.shape[1])
        for _ in range(self.epochs):
            idx = np.random.permutation(len(X))
            Xs, ys = X[idx], y[idx]
            for xi, target in zip(Xs, ys):
                pred = step(np.dot(xi, self.w))
                self.w += self.lr * (target - pred) * xi
        return self
models = {}
for lr in [0.01, 0.1]:
    model = Perceptron(lr=lr, epochs=20)
    model.fit(X, y)
    models[lr] = model
for lr in [0.01, 0.1]:
    model = models[lr]
    w = model.w
    plt.figure(figsize=(6,5))
    plt.scatter(X[:,0], X[:,1], c=y, cmap="bwr", alpha=0.6)
    x_min, x_max = X[:,0].min(), X[:,0].max()
    xs = np.linspace(x_min, x_max, 100)
    ys = -(w[0] + w[1]*xs) / w[2]
    plt.plot(xs, ys, 'k--', label="Decision Boundary")
    plt.xlabel("Rotational speed [rpm]")
    plt.ylabel("Torque [Nm]")
    plt.title(f"Perceptron (Learning Rate = {lr})")
    plt.legend()
    plt.grid()
    plt.show()
```

OUTPUT:

```
predictive_maintenance.csv(text/csv) - 531014 bytes, last modified: 12/04/2026 - 100% done
Saving predictive_maintenance.csv to predictive_maintenance (13).csv
```





OBSERVATION:

The model was trained using rotational speed and torque as features. It was observed that a learning rate of 0.1 led to faster convergence and a clearer separation boundary compared to a learning rate of 0.01.

BONUS TASK:

Take a dataset with 3 features (inputs) and 3 classes from Kaggle or any CSV file. Implement a multi-class perceptron using the one-vs-rest approach. Train the model, make predictions for new samples, and visualize the data in 3D. Analyze which features influence the classification the most and discuss any limitations of the perceptron on this dataset.

CODE:

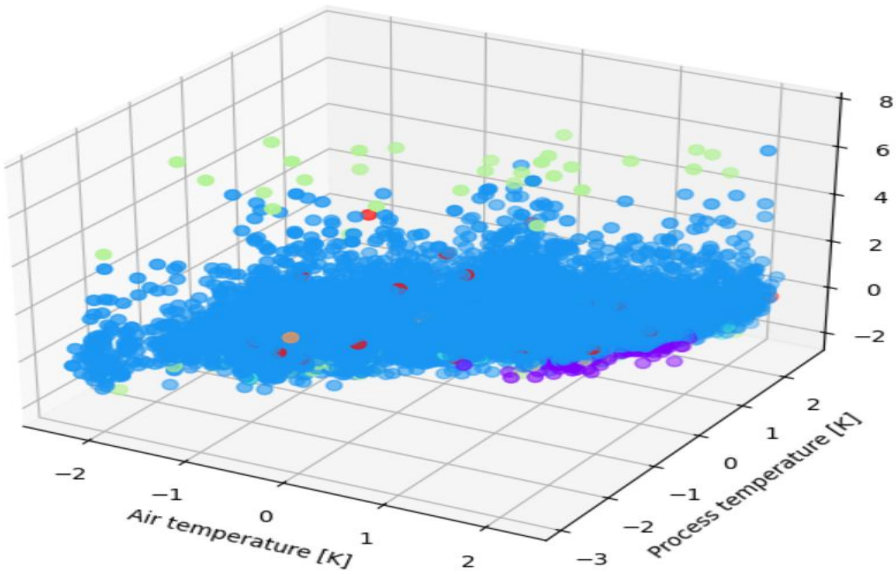
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from google.colab import files
uploaded = files.upload()
file_name = list(uploaded.keys())[0]
df = pd.read_csv(file_name)
print(df.head())
df = df.dropna()
target_col = "Failure Type"
features = ["Air temperature [K]", "Process temperature [K]", "Rotational speed [rpm]"]
X = df[features].values
y = df[target_col].values
classes = np.unique(y)
class_to_int = {c:i for i,c in enumerate(classes)}
y = np.array([class_to_int[i] for i in y])
print("Classes:", class_to_int)
X = (X - X.mean(axis=0)) / X.std(axis=0)
def step(z):
    return 1 if z > 0 else 0
class OvRPerceptron:
    def __init__(self, lr=0.1, epochs=20):
        self.lr = lr
        self.epochs = epochs
        self.weights = {}
    def train_binary(self, X, y_binary):
        X = np.insert(X, 0, 1, axis=1)
        w = np.zeros(X.shape[1])
        for _ in range(self.epochs):
            j = np.random.randint(len(X))
            pred = step(np.dot(X[j], w))
            w += self.lr * (y_binary[j] - pred) * X[j]
        return w
    def fit(self, X, y):
        self.classes = np.unique(y)
        for c in self.classes:
            y_binary = (y == c).astype(int)
            self.weights[c] = self.train_binary(X, y_binary)
    def predict(self, X):
        X = np.insert(X, 0, 1, axis=1)
        results = []
        for x in X:
            scores = {}
            for c, w in self.weights.items():
                scores[c] = np.dot(x, w)
            results.append(max(scores, key=scores.get))
        return np.array(results)
model = OvRPerceptron(lr=0.1, epochs=30)
```

```
model.fit(X, y)
preds = model.predict(X)
accuracy = np.mean(preds == y)
print("Training Accuracy:", accuracy)
new_sample = np.array([400, 330, 2500])
new_sample = (new_sample - X.mean(axis=0)) / X.std(axis=0)
pred = model.predict(new_sample.reshape(1, -1))
print("Predicted Class:", classes[pred[0]])
fig = plt.figure(figsize=(10,7))
ax = fig.add_subplot(111, projection='3d')

ax.scatter(
    X[:,0], X[:,1], X[:,2],
    c=y, cmap='rainbow', s=40)
ax.set_xlabel(features[0])
ax.set_ylabel(features[1])
ax.set_zlabel(features[2])
plt.title("3D View of Predictive Maintenance.")
plt.show()
importance = {}
for c, w in model.weights.items():
    importance[c] = w[1:]
print("Feature Influence:\n")
for c in importance:
    print("\nClass:", classes[c])
    print("Feature weights:", importance[c])
```

OUTPUT:

```
Choose files predictive_...ntenance.csv
predictive_maintenance.csv(text/csv) - 531014 bytes, last modified: 12/04/2026 - 100% done
Saving predictive_maintenance.csv to predictive_maintenance (2).csv
   UDI Product ID Type  Air temperature [K]  Process temperature [K]  \
0    1     M14860    M                298.1                    308.6
1    2     L47181    L                298.2                    308.7
2    3     L47182    L                298.1                    308.5
3    4     L47183    L                298.2                    308.6
4    5     L47184    L                298.2                    308.7

   Rotational speed [rpm]  Torque [Nm]  Tool wear [min]  Target Failure Type
0                    1551         42.8                0       0   No Failure
1                    1408         46.3                3       0   No Failure
2                    1498         49.4                5       0   No Failure
3                    1433         39.5                7       0   No Failure
4                    1408         40.0                9       0   No Failure
Classes: {'Heat Dissipation Failure': 0, 'No Failure': 1, 'Overstrain Failure': 2, 'Power Failure': 3, 'Random Failures': 4, 'Tool Wear Failure': 5}
Training Accuracy: 0.7285
Predicted Class: No Failure
```

3D View of Predictive Maintenance.




```
Feature Influence:


Class: Heat Dissipation Failure
Feature weights: [0. 0. 0.]

Class: No Failure
Feature weights: [-0.11548384 -0.10859091  0.23173755]

Class: Overstrain Failure
Feature weights: [0. 0. 0.]

Class: Power Failure
Feature weights: [0. 0. 0.]

Class: Random Failures
Feature weights: [0. 0. 0.]

Class: Tool Wear Failure
Feature weights: [0. 0. 0.]
```

ANALYZATION:

Rotational speed and torque have the greatest impact on the model because they are closely linked to the machine's mechanical load and operating state, while temperature-related features contribute less significantly.

LIMITATIONS OF PERCEPTRON:

- The perceptron can only create a linear boundary.
- In this dataset, normal and failure cases may overlap in feature space. The perceptron struggles in such cases because it cannot separate overlapping data well.
- The perceptron only gives binary or class predictions, not probabilities.
- Using one-vs-rest for multi-class classification can lead to: Imbalanced learning and Conflicting class boundaries